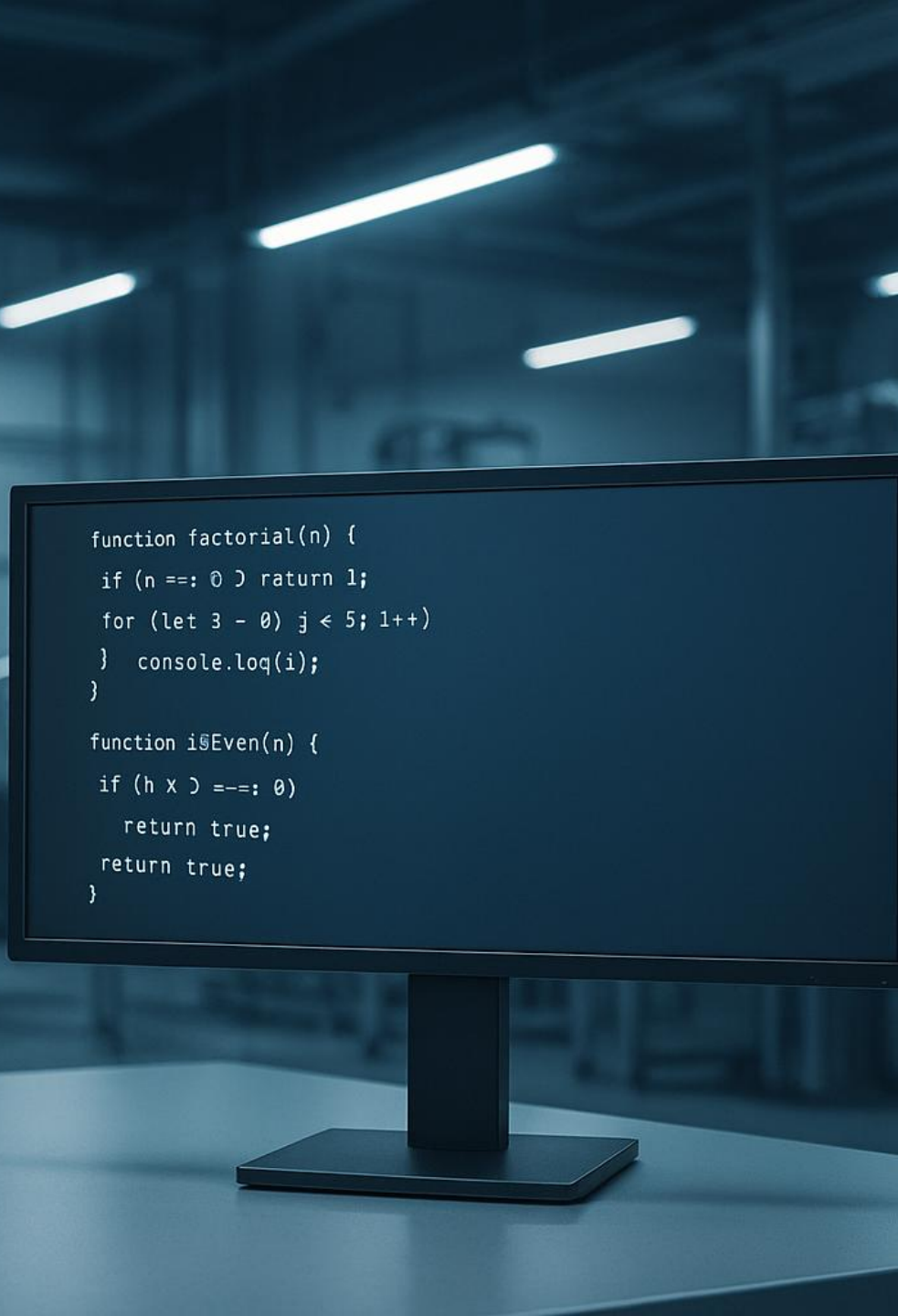
Firjan SENAI

Técnico

JavaScript

CONCEITOS • VARIÁVEIS • ESTRUTURAS CONDICIONAIS • LAÇOS DE REPETIÇÃO • ALGORITMOS



2025



Sumário

1. **Introdução à Lógica de Programação**
   O que é lógica de programação e por que aprender
2. **Tipos de Dados e Variáveis**
   Números, textos, booleanos e declaração de variáveis
3. **Operadores e Expressões**
   Aritméticos, relacionais e lógicos
4. **Estruturas Condicionais**
   Decisões com if, else e switch
5. **Laços de Repetição**
   Repetição com for, while e do...while
6. **Algoritmos**
   Representação e construção de soluções passo a passo
7. **Funções e Modularização**
   Organização do código em blocos reutilizáveis



03

Objetos

Desenvolvimento de Sistemas - Javascripts
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Introdução ao Objeto

Começando com Objetos

Objetos são estruturas de dados em JavaScript que permitem agrupar e organizar valores em pares chave: valor. Com objetos, é possível representar entidades completas, como um usuário, um produto ou um carro, armazenando diversas informações em uma única estrutura.

Objeto
É um tipo de dado em **JavaScript** utilizado para representar coleções de propriedades e seus respectivos valores. Cada valor é associado a uma chave (nome), facilitando o acesso e a manipulação dos dados.

Principais características dos Objetos:

- Armazena dados em pares chave: valor
- Permite acessar e modificar dados com . ou []
- Pode conter qualquer tipo de dado (strings, números, arrays, funções, etc.)
- Ideal para representar entidades reais (ex: um aluno, produto, veículo)

Primeiros comandos úteis:

- let usuario = { nome: 'Ana', idade: 25 } → Cria um objeto com duas propriedades
- console.log(usuario.nome) → Exibe "Ana" (valor da chave nome)
- usuario.email = 'ana@email.com' → Adiciona nova chave email ao objeto
- usuario.idade = 26 → Atualiza o valor da chave idade

Trabalhando com Objetos

Conceito básico

Objetos em JavaScript são estruturas que armazenam dados em pares de chave e valor. Eles são usados para representar entidades do mundo real com suas características.Cada chave (ou propriedade) é um identificador e cada valor pode ser de qualquer tipo: número, string, booleano, array, função, ou até outro objeto.

Para que serve

- Representar dados complexos de forma organizada
- Agrupar várias informações sobre uma entidade (ex: um usuário, um produto)
- Acessar e modificar dados de forma clara

Faça você mesmo

- Crie um arquivo chamado objetos.js
- Crie uma variáveis chamada usuario e atribua para essa variável um objeto como na imagem ao lado
- Busque informações de dentro do objeto como ilustrado na imagem
- No mesmo arquivo, crie um objeto representando um livro (com título, autor e ano) e exiba o título no console. Em seguida, altere o ano de publicação

```
objetos.js
objetos.js > ...
1  let usuario = {
2      nome: "Ana",
3      idade: 25,
4      email: "ana@email.com"
5  };
6
7  console.log(usuario.nome)
8  console.log(usuario.idade)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ...  powershell
PS C:\Users\Vitor\Desktop\servidor-express> node .\objetos.js
Ana
25
PS C:\Users\Vitor\Desktop\servidor-express>
```

Firjan SENAI

Trabalhando com Array de Objetos

Conceito básico

Um **array de objetos** em **JavaScript** é uma estrutura que combina dois conceitos importantes: listas (arrays) e agrupamentos de dados (objetos).
É muito usado quando precisamos armazenar **vários itens semelhantes**, como uma lista de usuários, produtos, livros, etc.

Faça você mesmo

- Crie um arquivo chamada arrayObjetos.js
- Crie um array chamado livros com três objetos contendo: título, autor e ano
- Exiba no console o título do primeiro livro
- Use filter() para retornar apenas os livros publicados após 2010

```
JS arrayObjetos.js ×

JS arrayObjetos.js > ...
1  let livros = [
2      { titulo: "Dom Casmurro", autor: "Machado de Assis", ano: 1899 },
3      { titulo: "O Hobbit", autor: "J.R.R. Tolkien", ano: 1937 },
4      { titulo: "Sapiens", autor: "Yuval Harari", ano: 2011 }
5  ];
6
7  console.log(livros[0].titulo); // Dom Casmurro
8
9  livros.forEach(livro => console.log(livro.titulo));
10
11 let recentes = livros.filter(livro => livro.ano > 2010);
12 console.log(recentes);
13
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\Vitor\Desktop\servidor-express> node .\arrayObjetos.js
Dom Casmurro
Dom Casmurro
O Hobbit
Sapiens
[ { titulo: 'Sapiens', autor: 'Yuval Harari', ano: 2011 } ]
PS C:\Users\Vitor\Desktop\servidor-express>
```

Método find()

O método find() percorre um vetor e retorna o **primeiro elemento que atende a uma condição**. Em um vetor de objetos, podemos utilizá-lo para localizar um produto pelo seu código, por exemplo.

Durante a busca, o parâmetro produto representa o objeto que está sendo analisado. A expressão produto.id === 2 verifica se esse objeto possui o ID procurado. Quando a condição é verdadeira, o find() retorna o objeto e encerra a busca.

Quando nenhum elemento atende à condição, o retorno é undefined. Por isso, devemos verificar o resultado antes de acessar suas propriedades.

```
const produtos = [
  { id: 1, nome: "Mouse", preco: 80 },
  { id: 2, nome: "Teclado", preco: 150 },
  { id: 3, nome: "Monitor", preco: 900 }
];

const produtoEncontrado = produtos.find(produto => produto.id === 2);

console.log(produtoEncontrado);
// { id: 2, nome: "Teclado", preco: 150 }
```

```
const resultado = produtos.find(produto => produto.id === 99);

if (resultado !== undefined) {
  console.log(resultado.nome);
} else {
  console.log("Produto não encontrado");
}
```

Operador de espalhamento

Spread Operator (...)

O operador de espalhamento, representado por **três pontos (...)**, permite espalhar as propriedades de um objeto ou os elementos de um vetor dentro de uma nova estrutura.

Com objetos, podemos utilizá-lo para **criar um novo objeto a partir de outro**, preservando suas propriedades e alterando apenas os dados necessários.

Nesse exemplo, ...produto copia as propriedades do objeto original para o novo objeto. Em seguida, preco: 70 define um novo valor para o preço.

A ordem das propriedades faz diferença: quando uma propriedade aparece mais de uma vez, prevalece o último valor informado.

```
const produto = {
  id: 1,
  nome: "Mouse",
  preco: 80
};

const produtoAtualizado = {
  ...produto,
  preco: 70
};

console.log(produto);
// { id: 1, nome: "Mouse", preco: 80 }

console.log(produtoAtualizado);
// { id: 1, nome: "Mouse", preco: 70 }
const exemplo = {
  preco: 70,
  ...produto
};

console.log(exemplo.preco); // 80
```

Operador de espalhamento

Spread em vetores

Também podemos utilizar o spread para criar um novo vetor contendo os elementos de um vetor existente e acrescentar novos elementos.

```
const nomes = ["Ana", "Carlos"];

const novosNomes = [...nomes, "Mariana"];

console.log(nomes);
// ["Ana", "Carlos"]

console.log(novosNomes);
// ["Ana", "Carlos", "Mariana"]
```

Exercício 1

Dados de um produto

Crie um objeto chamado produto com estas propriedades:

Propriedade	Tipo	Valor
id	Número	101
nome	Texto	Teclado
preco	Número	150
estoque	Número	8
categoria	Texto	Periféricos

Depois:

- Calcule o valor total desse produto em estoque: preço × quantidade.
- Exiba: Teclado — R$ 150.00 — Estoque: 8.
- Simule a venda de duas unidades, atualizando o estoque.
- Exiba “Produto disponível” se ainda houver unidades ou “Produto esgotado” caso contrário.

Dica: utilize toFixed(2) para exibir o preço com duas casas decimais.

Exercício 2

Vetor dentro de um objeto

Crie um objeto chamado cliente com:

- id: 1.
- nome: Carlos Oliveira.
- endereco: outro objeto, contendo rua, numero e cidade, com valores escolhidos por você.
- interesses: um vetor com três textos representando categorias de produtos.

Depois:

- Exiba somente a cidade do cliente.
- Exiba o primeiro interesse.
- Altere a cidade.
- Percorra os interesses e exiba cada um no console.

Exercício 3

Vetor de objetos

Acesse o arquivo disponibilizado. Ele contém dez produtos com id, nome, categoria, preco, estoque e ativo. Há produtos sem estoque e produtos inativos para permitir diferentes consultas.

Acessando e percorrendo os produtos:

- Exiba o nome do primeiro produto.
- Exiba o preço do terceiro produto.
- Exiba a quantidade de produtos cadastrados.
- Utilize um laço ou forEach() para exibir todos os produtos no formato:
    - 1 — Mouse sem fio — R$ 89.90

Filtrando o catálogo
Utilize filter() para criar vetores separados contendo:

- Apenas produtos da categoria "Acessórios".
- Apenas produtos com preço inferior a R$ 100.
- Apenas produtos com estoque maior que zero.
- Apenas produtos ativos e com estoque maior que zero.

Exiba cada resultado. Estar ativo e ter estoque são condições diferentes: o produto pode estar cadastrado como ativo mesmo sem unidades disponíveis.

Exercício 4 - Continuação do anterior

Buscando um produto pelo ID

Crie uma função chamada buscarProdutoPorId(id).

A função deve retornar o objeto que possui o ID recebido. Utilize find().

Teste a função com:

- ID 3, que existe.
- ID 99, que não existe.

Ao exibir o resultado, verifique se o produto foi encontrado. Caso contrário, mostre Produto não encontrado.

Exercício 5 - Continuação do anterior

Preparando informações para uma lista

Utilize map() para criar:

- Um vetor chamado nomesProdutos, contendo somente os nomes.
- Um vetor chamado resumosProdutos, contendo objetos com apenas id, nome e preco.

Não altere o vetor produtos.

Exercício 6 - Continuação do anterior

Criando uma versão atualizada de um produto

Localize o produto de ID 2 e crie um novo objeto chamado produtoAtualizado:

- Mantenha todas as propriedades do produto encontrado.
- Altere somente o preço para 219.90.

- Utilize o operador de espalhamento: `...`.

Exiba o produto original e o novo objeto. O preço do original deve continuar sendo 249.90.

Exercício 7 - Continuação do anterior

Atualizando um produto dentro do vetor

Crie um novo vetor chamado produtosAtualizados, utilizando map().

Nesse novo vetor:

- O produto de ID 5 deve ter estoque igual a 10.
- As demais informações desse produto devem ser preservadas.
- Os demais produtos devem ser mantidos.
- O vetor original deve continuar com estoque 12 para o produto de ID 5.

Utilize o ID para identificar o produto, mesmo que a posição dele no vetor mude.

Exercício 8 - Continuação do anterior

Desafio final — inclusão e remoção

Partindo novamente do vetor original:

- Crie um objeto para um novo produto, seguindo a mesma estrutura e utilizando o ID 11.
- Crie produtosComNovo, contendo os produtos existentes e o novo produto. Utilize ....
- A partir de produtosComNovo, crie produtosSemRemovido, removendo o produto de ID 6 com filter().
- Confira as quantidades: o vetor original deve ter 10, o vetor após inclusão 11 e o vetor após remoção 10 produtos.

Firjan SENAI